Gradient Descent Algorithm Design

Luke Catalano

2026-03-09

Objective

Iterative optimization algorithms are used behind the scenes in nearly every machine learnign framework. The objective of this workbook is to combine my experiences in AM230, Econ 217, and Econ 294A by implementing a gradient descent algorithm from scratch on the credit dataset to predit the credit score of indiviuduals with "relative accuracy". Pre-packaged regularized regression models achieve 90% weighted accuracy on the 80 test observations, so i'm hoping to beat this through my own custom algorithm.

Dataframe Structure

```
##    Income Limit Rating Cards Age Education Own Student Married Region Balance
## 1  14.891  3606    283     2  34        11  No      No     Yes  South     333
## 2 106.025  6645    483     3  82        15 Yes     Yes     Yes   West     903
## 3 104.593  7075    514     4  71        11  No      No      No   West     580
```

Numeric Conversion

```r
Credit <- Credit %>%
  mutate(
    Own = if_else(Own == "Yes", 1, 0),
    Student = if_else(Student == "Yes", 1, 0),
    Married = if_else(Married == "Yes", 1, 0),
    Region = as.integer(Region))
head(Credit,3)
```

```
##    Income Limit Rating Cards Age Education Own Student Married Region Balance
## 1  14.891  3606    283     2  34        11   0       0       1      2     333
## 2 106.025  6645    483     3  82        15   1       1       1      3     903
## 3 104.593  7075    514     4  71        11   0       0       0      3     580
```

Select Outcome Variable, Scale the Covariates

```r
y <- as.matrix(Credit$Rating)
X <- Credit %>%
  select(-Rating)

X <- scale(X)
head(round(X,3),3)
```

```
##   Income  Limit  Cards    Age Education    Own Student Married Region Balance
## 1 -0.861 -0.489 -0.698 -1.256    -0.784 -1.034  -0.333   0.794 -0.011  -0.407
## 2  1.725  0.827  0.031  1.527     0.496  0.964   2.996   0.794  1.398   0.833
## 3  1.685  1.014  0.760  0.889    -0.784 -1.034  -0.333  -1.256  1.398   0.130
```

Matrix Construction

Adding the intercept column to our X matrix completes the regression setup.

$$\begin{bmatrix} 1 & X_{21} & X_{31} & \dots & X_{(m+1)1} \\ 1 & X_{22} & X_{32} & \dots & X_{(m+1)2} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & X_{2n} & X_{3n} & \dots & X_{(m+1)n} \end{bmatrix}_{(m+1)\times n}$$

Problem formulation and objective

We seek to achieve local convergence with a gradient descent algorithm, on a 2024 M4 Pro Macbook, ran locally. This is a much simpler task than global convergence, and our small sample size (n = 400) and feature space of (m = 10) simplify this problem greatly. Because gradient descent is an iterative algorithm with sequential parameter updates, parallelization provides limited benefits for small datasets. We begin with the following model specification:

$$y = X\beta$$

Where y is a vector of credit ratings, X is a matrix of feature values, and $\beta \in \mathbb{R}^{m+1}$ is the vector of regression coefficients. By iteratively updating our coefficient vector optimally, we decrease the following loss function:

$$L(\beta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - X_i\beta)^2$$

Gradient Descent Framework

Each update to our coefficient vector follows a descent direction criteria, ensuring we always move toward the optimal β^*, the parameter vector that minimizes our error.

$$\beta_{k+1} = \beta_k - \alpha p_k$$

where α is a step size, determining how far of a step we take in m-dimensional space when attempting to decrease the loss function. For this problem formulation, we will keep α contant at 0.01. p_k is a direction equivalent to the negative gradient of the loss function at step k ($p_k = -\nabla L(\beta_k)$). The gradient of the loss function represents the direction of steepest increase. Therefore, updating parameters in the negative gradient direction produces the steepest decrease in the loss. These iterative updates ensure we are always traveling in the direction that decreases the loss function the fastest. We define the gradient of the loss function at iteration k as follows:

$$\nabla L(\beta_k) = \frac{2}{n}\sum_{i=1}^{n}(y_i - X_i\beta_k) \times -X_i = \frac{2}{n}X^T(X\beta_k - y)$$

Memory Usage Assuming 8 bytes per entry:

With 11 total parameters, and 8 bytes per entry, our β parameter vector only takes up 88 bytes. Our gradient $\nabla L(\beta)$ is constructed of two matrices multiplies together. X^T is an 11×400 matrix, and $(X\beta_k - y)$ is a 400×1 vector, which results in a gradient matrix with $11 \times 1 = 11$ total entries, which equates to the same 88 bytes of storage required to store the parameter vector. The storage and computation of our gradient computationally cheap. Gradient Descent does not require the computation of a Hessian. It is worth noting, however, that second order methods would only require $8(11)^2 = 968$ bytes, making second order methods computationally reasonable for a problem of this size.

Setup

```r
# Initialize our parameter vector with random values pulled from a normal distribution
set.seed(0)
m <- ncol(X) # Number of coefficients
n <- nrow(X) # Number of observations
a <- 0.1 # step size = 0.01
B <- as.matrix(rnorm(m, 0, 1)) # Initialization vector from normal dist.
error_vector <- c()
error <- (1/n)*sum((y - X%*%B)^2) # error function
error_diff <- 1 # Difference in error between iterations, used for stopping the loop
iter <- 1 # iteration counter
```

Implementation

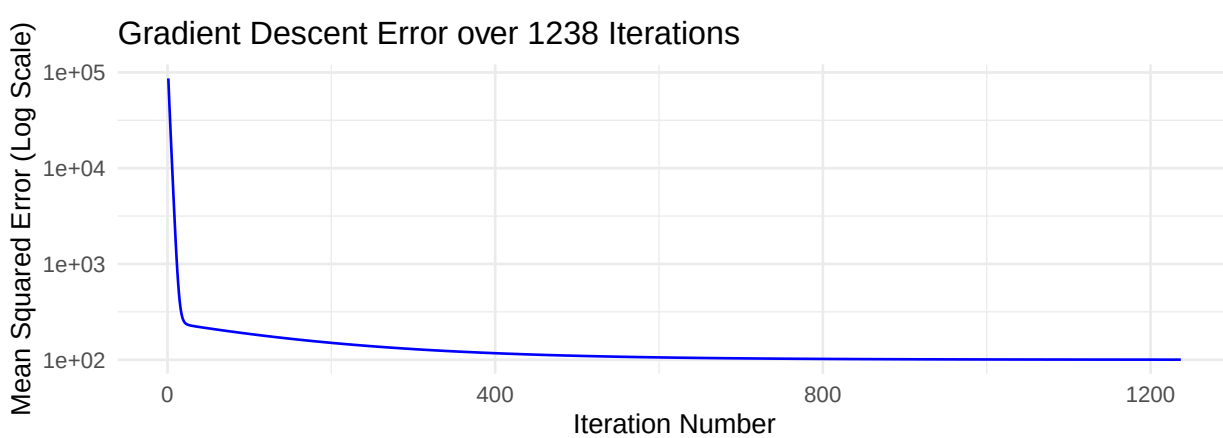
```r
source("methods/mse_loss.R") # Calculates the error using the MSE loss function
source("methods/grad_mse_loss.R") # Returns Negative Gradient of MSE loss function

while (error_diff > 0.001){

  # Update and Error calculation
  gradient <- grad_mse_loss(y, X, B) # Gradient of param vector
  B <- B - a * gradient # Gradient Descent
  error <- mse_loss(y, X, B) # Error recalculation
  error_vector[iter] <- error # Store the error for this iteation

  # Evaluate stopping criteria
  if(iter > 1){
    error_diff <- abs(error_vector[iter]-error_vector[iter-1])
  }
  else{
    error_diff <- 1
  }
  iter <- iter + 1 # increase iteration
}
cat("Grad Descent Successfully Converged in", iter,
    "iterations with an error of", error, sep=" ")
```

```
## Grad Descent Successfully Converged in 1238 iterations with an error of 100.1739
```



Model testing

We will now train our model on a base set of training data and evaluate its performance on unseen testing data. Our benchmark to beat is roughly 90% accuracy from basic regularized regression.

```
p <- 0.8 # Split Parameter
split <- round(p*nrow(X))

m <- ncol(X) # Number of coefficients

X_train <- X[1:split, ]
X_test <- X[(split+1):nrow(X), ]
y_train <- as.matrix(y[1:split])
y_test <- as.matrix(y[(split+1):nrow(X)])

# Setup
max_iter <- 10000
set.seed(0)
n <- nrow(X_train) # Number of training observations
a <- 0.1 # step size = 0.1
B <- as.matrix(rnorm(m, 0, 1)) # Initialization vector from normal dist.
error_vector <- numeric()
error <- mse_loss(y_train, X_train, B) # MSE loss function
error_diff <- 1 # Difference in error between iterations, used for stopping the loop
iter <- 1 # iteration counter

# Gradient Descent
while (error_diff > 0.001 && iter < max_iter){

  # Update and Error calculation
  gradient <- grad_mse_loss(y_train, X_train, B) # Gradient of param vector
  B <- B - a * gradient # Gradient Descent
  error <- mse_loss(y_train, X_train, B) # Error recalculation
  error_vector[iter] <- error # Store the error for this iteation

  # Evaluate stopping criteria
  if(iter > 1){
    error_diff <- abs(error_vector[iter]-error_vector[iter-1])
  } else{ error_diff <- 1 }
  iter <- iter + 1 # increase iteration
}
cat("Grad Descent Successfully Converged in", iter,
    "iterations with an error of", error, sep=" ")
```

```
## Grad Descent Successfully Converged in 1256 iterations with an error of 102.4063
```

```
# Model Testing
predictions <- X_test %*% B

# Weighted Absolute Percent Error
WAPE <- sum(abs(y_test - predictions))/sum(y_test)
Accuracy <- 1 - WAPE
cat("Model Accuracy: ", round(Accuracy*100,2), "%", sep="")
```

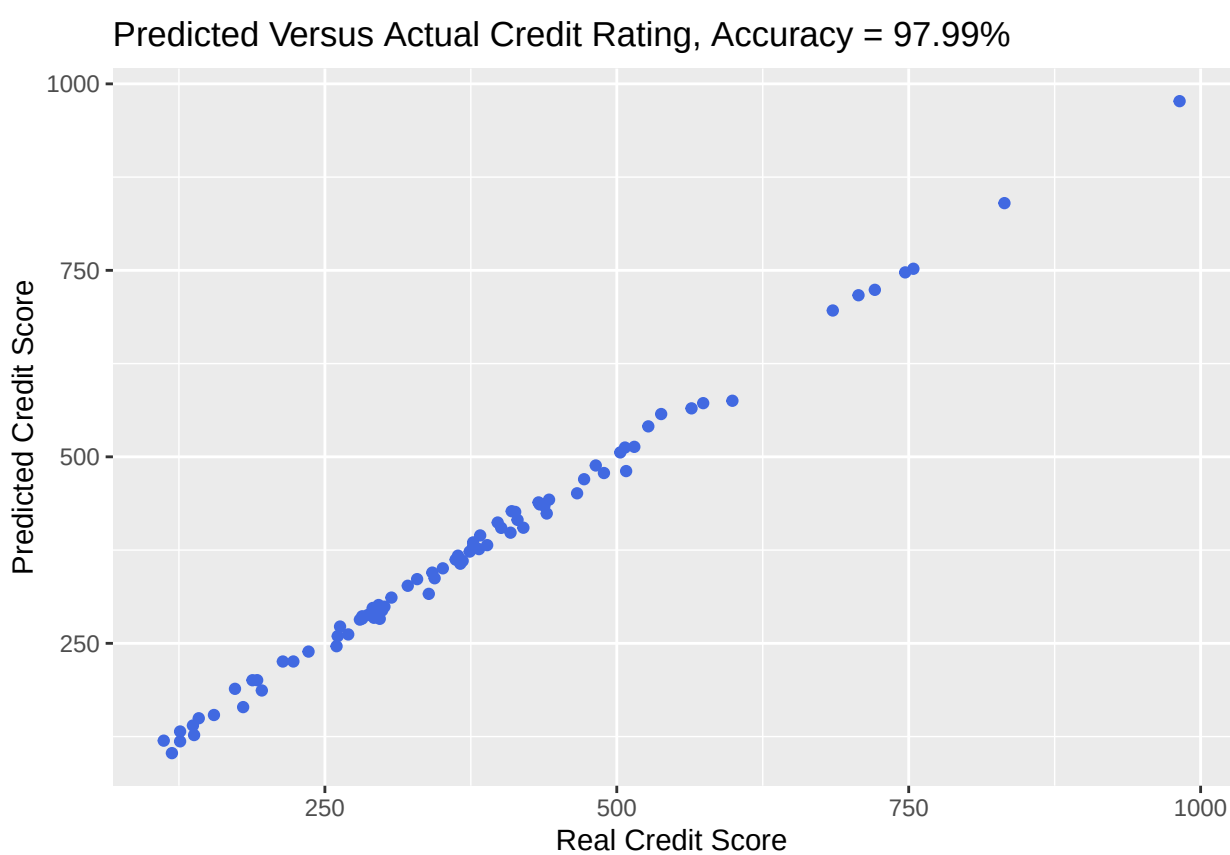
```
## Model Accuracy: 97.99%
```

Futher Testing

```
X_Test_df <- as.data.frame(X_test)
X_Test_df$Credit_Score <- as.data.frame(y_test)
X_Test_df$Prediction <- as.data.frame(predictions)
X_Test_df$error <- abs(X_Test_df$Credit_Score - X_Test_df$Prediction)

X_Test_df <- data.frame(
  real = as.numeric(y_test),
  preds = as.numeric(predictions)
)

ggplot(X_Test_df, mapping = aes(x=real, y=preds))+
  geom_point(color='royalblue')+
  labs(
    title = paste("Predicted Versus Actual Credit Rating, Accuracy = ",
                  round(Accuracy*100,2), "%", sep=""),
    x = "Real Credit Score",
    y = "Predicted Credit Score"
  )
```



We have successfully achieved an accuracy of ~98.00%, significantly outperforming the prepackaged regularized regression models.